# Biblioteca Híbrida — Como Funciona por Dentro

Entenda como uma aplicação híbrida acessa câmera, GPS, WhatsApp, SMS e compartilhamento do celular usando HTML, CSS e JavaScript — sem precisar escrever código nativo Android ou iOS.

Câmera · Geolocalização · WhatsApp · SMS · Compartilhar · Fluxo Integrado

## 1. O que é uma Aplicação Híbrida?

> **Conceito-chave:** uma aplicação híbrida é um aplicativo mobile construído com **tecnologias web** (HTML, CSS e JavaScript) que roda dentro de um contêiner nativo chamado **WebView**. O WebView é uma janela que exibe sua página web *dentro* do aplicativo, mas com acesso a recursos do celular.

Diferente do navegador comum (Chrome, Safari), o WebView de um app híbrido pode ter acesso a **câmera, GPS, contatos, arquivos e microfone** — porque o app nativo que o empacota concede essas permissões.

### As 3 camadas de uma aplicação híbrida

**CAMADA 1**

#### Interface (Web)

HTML, CSS e JavaScript que o usuário vê. É aqui que ficam botões, formulários e chamadas às APIs Web.

**CAMADA 2**

#### Ponte (Bridge)

Camada que traduz chamadas do JavaScript para o sistema operacional. É aqui que frameworks como Capacitor, Cordova ou PhoneGap atuam.

**CAMADA 3**

#### Sistema Operacional

Android ou iOS. Fornece os sensores, câmeras, GPS e APIs nativas. Só aceita comandos da Bridge, não do JS diretamente.

### 🔄 Como o JavaScript conversa com o hardware




> ⚙️ **Importante:** nem toda API Web precisa da Bridge. Algumas (como `wa.me` e `sms:`) são apenas *links especiais* que o sistema operacional reconhece e abre no app correspondente.

## 🔌 2. Como o app híbrido acessa os sensores do celular

Existem **duas grandes famílias** de acesso a recursos do celular em aplicações híbridas:

### 🌐 Família 1 — APIs Web padrão (HTML5)

São APIs definidas pelo **W3C** e suportadas pelos navegadores modernos. Não precisam de instalação. Exemplos:

| API Web | O que acessa | Comando principal |
|---|---|---|
| `navigator.geolocation` | GPS / Localização | `getCurrentPosition()` |
| `navigator.mediaDevices` | Câmera e microfone | `getUserMedia()` |
| `input type="file"` | Câmera, galeria e arquivos | `capture="environment"` |
| `navigator.share` | Compartilhamento nativo | `navigator.share({...})` |
| `navigator.vibrate` | Vibração | `navigator.vibrate(200)` |
| `localStorage` | Armazenamento local | `setItem() / getItem()` |
| `Notification` | Notificações do sistema | `new Notification()` |

### 🌉 Família 2 — Plugins nativos (via Bridge)

Quando a API Web não é suficiente (ex.: ler contatos, usar Bluetooth clássico, acessar arquivos do sistema), usamos **plugins nativos** fornecidos pelos frameworks híbridos:

| Framework | Plugin / recurso | O que acessa |
|---|---|---|
| **Capacitor** | `@capacitor/camera` | Câmera nativa com controle total |
| **Capacitor** | `@capacitor/geolocation` | GPS com precisão nativa |
| **Capacitor** | `@capacitor/share` | Compartilhamento nativo |
| **Capacitor** | `@capacitor/filesystem` | Arquivos do aparelho |
| **Cordova** | `cordova-plugin-camera` | Câmera (legado) |
| **Cordova** | `cordova-plugin-geolocation` | Geolocalização (legado) |

> ⚠️ **Atenção:** toda API sensível exige **HTTPS** ou **localhost**. Em HTTP puro, o navegador bloqueia geolocalização, câmera (via getUserMedia) e compartilhamento nativo. Câmera via `input file`, WhatsApp e SMS funcionam mesmo em HTTP.

## 📷 3. Exemplo 1 — Câmera (input type="file")

> 💡 **Como funciona:** não usamos `getUserMedia`. Usamos um `<input type="file">` com o atributo `capture`. O sistema operacional abre o **app nativo de câmera** e devolve a foto como arquivo.

### 🎯 Comandos principais

| Comando | Função |
|---|---|
| `accept="image/*"` | Aceita apenas imagens |
| `capture="environment"` | Abre câmera traseira |
| `capture="user"` | Abre câmera frontal (selfie) |
| `input.files[0]` | Pega o arquivo escolhido |
| `new FileReader()` | Lê o arquivo como dados |
| `reader.readAsDataURL()` | Converte a imagem em base64 |

### 💻 Estrutura do código

```
<input type="file" accept="image/*" capture="environment">

<script>
  const input = document.querySelector('input');
  input.addEventListener('change', function() {
    const arquivo = input.files[0];
    const leitor = new FileReader();
    leitor.onload = function(e) {
      document.getElementById('preview').src = e.target.result;
    };
    leitor.readAsDataURL(arquivo);
  });
</script>
```

> 🔐 **HTTPS?** Não exige. O `input file` funciona em qualquer origem, porque o arquivo é escolhido diretamente pelo usuário.

## 📍 4. Exemplo 2 — Geolocalização (GPS)

> 💡 **Como funciona:** a API `navigator.geolocation` pergunta ao sistema operacional a posição atual. O S.O. usa GPS + Wi-Fi + torres de celular para calcular latitude, longitude e precisão.

### 🎯 Comandos principais

| Comando | Função |
|---|---|
| `navigator.geolocation.getCurrentPosition()` | Obtém a posição uma vez |
| `watchPosition()` | Monitora a posição continuamente |
| `pos.coords.latitude` | Latitude |
| `pos.coords.longitude` | Longitude |
| `pos.coords.accuracy` | Precisão em metros |
| `enableHighAccuracy: true` | Força uso do GPS |
| `timeout: 10000` | Tempo máximo de espera |

### 💻 Estrutura do código

```
navigator.geolocation.getCurrentPosition(
  function(pos) {
    console.log("Lat:", pos.coords.latitude);
    console.log("Lon:", pos.coords.longitude);
    console.log("Precisão:", pos.coords.accuracy, "m");
  },
  function(erro) {
    console.log("Erro:", erro.message);
  },
  { enableHighAccuracy: true, timeout: 10000 }
);
```

> ⚠️ **HTTPS obrigatório.** Em `file://` ou `http://`, o navegador bloqueia a geolocalização. Use **https**, **localhost** ou ferramentas como **ngrok**.

## 📏 5. Exemplo 3 — Cálculo de Distância entre Dois Pontos

> 💡 **Como funciona:** usamos a **Fórmula de Haversine** para calcular a distância entre duas coordenadas geográficas na superfície da Terra. Não é uma API do celular — é pura matemática em JavaScript.

### 🎯 Fórmula de Haversine

```
const R = 6371; // raio da Terra em km
const dLat = (latB - latA) * Math.PI / 180;
const dLon = (lonB - lonA) * Math.PI / 180;

const a =
  Math.sin(dLat/2) ** 2 +
  Math.cos(latA * Math.PI/180) *
  Math.cos(latB * Math.PI/180) *
  Math.sin(dLon/2) ** 2;

const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
const distancia = R * c; // km
```

### 📊 Comandos usados

| Comando | Função |
| --- | --- |
| `parseFloat()` | Converte texto em número decimal |
| `Math.sin() / Math.cos()` | Funções trigonométricas |
| `Math.atan2()` | Arco tangente de dois valores |
| `Math.sqrt()` | Raiz quadrada |
| `.toFixed(2)` | Formata com 2 casas decimais |

> 💼 **Uso real:** apps de entrega (iFood, Uber), check-in em eventos, cálculo de frete, proximidade de lojas.

## 💬 6. Exemplo 4 — Acionar WhatsApp por Link

> 💡 **Como funciona:** o WhatsApp registra no sistema operacional um *esquema de URL* ( `https://wa.me/` ). Quando o navegador abre esse link, o S.O. redireciona automaticamente para o app do WhatsApp.

### 🎯 Formato do link

```
https://wa.me/5532999999999?text=Ol%C3%A1%2C%20tudo%20bem%3F

// Estrutura:
// https://wa.me/NUMERO?text=MENSAGEM_URL_ENCODED
// Número deve ter DDI + DDD, sem espaços nem traços
```

### 📊 Comandos usados

| Comando | Função |
| --- | --- |
| `.replace(/\D/g, "")` | Remove tudo que não for número |
| `encodeURIComponent()` | Codifica a mensagem para URL |
| `window.open(url, "_blank")` | Abre em nova aba/app |

### 💻 Código essencial

```
const numero = "5532999999999";
const msg = "Olá, tudo bem?";
const url = "https://wa.me/" + numero + "?text=" + encodeURIComponent(msg);
window.open(url, "_blank");
```

> 🔐 **HTTPS?** Não exige. Funciona em qualquer origem, porque é apenas um link.

## 📩 7. Exemplo 5 — Enviar SMS

> 💡 **Como funciona:** usamos o esquema `sms:` . O sistema operacional reconhece esse formato e abre o app nativo de mensagens já com número e texto pré-preenchidos.

### 🎯 Formato do link

```
// Android:
sms:32999999999?body=Olá,%20tudo%20bem?

// iOS (usa & em vez de ?):
sms:32999999999&body=Olá,%20tudo%20bem?
```

### 📊 Comandos usados

| Comando | Função |
|---|---|
| `/iPad\|iPhone\|iPod/.test(navigator.userAgent)` | Detecta iOS |
| `window.location.href = url` | Abre o app de SMS |

### 💻 Código essencial

```
const numero = "32999999999";
const msg = "Olá! Vim pelo site.";
const ehIOS = /iPad|iPhone|iPod/.test(navigator.userAgent);
const sep = ehIOS ? "&" : "?";
const url = "sms:" + numero + sep + "body=" + encodeURIComponent(msg);
window.location.href = url;
```

> ⚠️ O SMS **não é enviado automaticamente**. O usuário precisa tocar em "Enviar" no app

de mensagens. Isso é uma proteção contra spam.

## 📤 8. Exemplo 6 — Compartilhar (Web Share API)

💡 **Como funciona:** a **Web Share API** abre o menu nativo de compartilhamento do celular (WhatsApp, Telegram, e-mail, etc.). É uma API padronizada pelo W3C, suportada por Chrome Android e Safari iOS.

### 🎯 Comandos principais

| Comando | Função |
|---|---|
| `navigator.share(dados)` | Abre o menu nativo |
| `navigator.canShare(dados)` | Verifica se o conteúdo pode ser compartilhado |
| `dados.title` | Título do compartilhamento |
| `dados.text` | Texto |
| `dados.url` | URL |
| `dados.files` | Arquivos (imagens, PDFs) |

### 💻 Código essencial

```
navigator.share({
  title: "Promoção Especial",
  text: "Aproveite 30% de desconto!",
  url: "https://meusite.com"
})
.then(() => console.log("Compartilhado!"))
.catch((erro) => console.log("Cancelado:", erro));
```

⚠️ **HTTPS obrigatório.** A Web Share API só funciona em HTTPS ou localhost.

## 🚀 9. Exemplo 7 — Fluxo Integrado (GPS + WhatsApp)

> 💡 **Como funciona:** combinamos **3 APIs** em um único fluxo: leitura de campos ( `input` ), captura de GPS ( `geolocation` ) e envio por WhatsApp ( `wa.me` ).

### 🔄 Fluxo passo a passo




### 💻 Código resumido

```
// 1. Captura GPS
navigator.geolocation.getCurrentPosition(function(pos) {
  const lat = pos.coords.latitude.toFixed(6);
  const lon = pos.coords.longitude.toFixed(6);

  // 2. Monta a mensagem
  const nome = document.getElementById("nome").value;
  const pedido = document.getElementById("pedido").value;
  const msg = "Olá! Meu nome é " + nome +
              ". Pedido: " + pedido +
              ". Localização: https://www.google.com/maps?q=" + lat + "," + lon;

  // 3. Envia por WhatsApp
  const url = "https://wa.me/5532999999999?text=" + encodeURIComponent(msg);
  window.open(url, "_blank");
});
```

> 💼 **Uso real:** apps de delivery, assistência técnica, chamados de emergência, pedidos em feiras e food trucks.

## 📊 10. Tabela Resumo — Todas as APIs

| Exemplo | API / Comando | HTTPS? | Sensor acessado |
|---|---|---|---|
| 📷 Câmera | `input file + capture` | Não | Câmera nativa |
| 📍 GPS | `navigator.geolocation` | Sim | GPS + Wi-Fi |
| 📏 Distância | Haversine (matemática) | Não | Nenhum |

| Exemplo | API / Comando | HTTPS? | Sensor acessado |
|---|---|---|---|
| WhatsApp | `https://wa.me/` | Não | App externo |
| SMS | `sms:numero?body=` | Não | App externo |
| Compartilhar | `navigator.share` | Sim | Menu nativo |
| Integrado | GPS + WhatsApp | Sim (por GPS) | GPS + app externo |

## 11. Questionário de Fixação (15 Questões)

Responda às questões e clique em "Verificar Respostas" para ver sua pontuação:

1 **O que é uma aplicação híbrida?**

- Um app escrito apenas em Kotlin e Java
- **Um app feito com HTML/CSS/JS rodando dentro de um WebView**
- Um app que só funciona no navegador do computador
- Um sistema operacional para celulares

2 **Qual é a função do WebView em uma aplicação híbrida?**

- **Renderizar HTML/CSS/JS dentro do app nativo**
- Compilar código Java para Android
- Armazenar arquivos no servidor
- Bloquear o acesso à internet

3 **Qual API Web é usada para capturar a localização do usuário?**

- navigator.camera
- **navigator.geolocation**
- navigator.share
- navigator.storage

**4** **Como abrimos a câmera traseira do celular em um app híbrido?**

- Usando capture="user"
- **Usando capture="environment"**
- Usando capture="rear"
- Usando capture="back"

**5** **Qual comando converte a imagem capturada em uma string base64?**

- reader.readAsText()
- **reader.readAsDataURL()**
- reader.readAsBinary()
- reader.readAsImage()

**6** **Por que a API de geolocalização exige HTTPS?**

- Porque é mais bonito
- **Por segurança: a localização é um dado sensível**
- Porque o GPS só funciona à noite

- Porque o navegador não sabe ler latitude

7 **Qual é o formato correto do link do WhatsApp?**

- https://whatsapp.com/send
- **https://wa.me/NUMERO?text=MENSAGEM**
- whatsapp://numero
- https://zap.me/NUMERO

8 **No link do WhatsApp, para que serve o encodeURIComponent()?**

- **Codificar a mensagem para formato seguro de URL**
- Aumentar a velocidade da internet
- Traduzir a mensagem para inglês
- Apagar a mensagem após o envio

9 **Qual é a diferença do link de SMS entre Android e iOS?**

- Não há diferença
- **Android usa ? e iOS usa & para separar o body**
- iOS não aceita SMS
- Android só envia SMS em inglês

10 **Qual API abre o menu nativo de compartilhamento do celular?**

- navigator.send

- **navigator.share**
- navigator.open
- navigator.menu

11 **Qual fórmula é usada para calcular a distância entre duas coordenadas?**

- Fórmula de Pitágoras
- **Fórmula de Haversine**
- Fórmula de Bhaskara
- Fórmula de Newton

12 **Qual API Web NÃO exige HTTPS para funcionar?**

- navigator.geolocation
- navigator.share
- **input type="file" capture**
- navigator.mediaDevices

13 **Um aplicativo híbrido pode acessar câmera, GPS e compartilhamento nativo sem escrever código nativo Android ou iOS.**

- ✅ **Verdadeiro**
- ❌ Falso

14 **Frameworks como Capacitor e Cordova criam uma "ponte" entre o JavaScript e as APIs nativas do sistema operacional.**

- ✅ **Verdadeiro**
- ❌ Falso

15 **O link sms: envia a mensagem automaticamente assim que o usuário clica, sem precisar de confirmação.**

- ○ ✅ Verdadeiro
- ◉ ❌ **Falso**

✅ Verificar Respostas  🔄 Limpar Respostas

## 📊 Resultado

# 15 / 15

**Aproveitamento: 100%**